



# SeeStars



MEMBRE DU PROJET

ADAM

MATHIEU

ROCHES

ENCADRANT

M BEVERINI

# **SOMMAIRES**

- I. GUI
- II. AUTHENTICATION
- III. Satellite
- IV. Interface
- V. MainLoc
- VI. Start
- VII. Sqlitecmd
- VIII. Commandes\_guides
- IX. Affichage\_satellite

## I. GUI

Il s'agit du point d'entrée de cette application. La plus courte des classes, on vient y choisir le type de base de donnée à utiliser, distante ou locale dans le main, qui est la première exécutée lors du lancement du programme .

Elle est composée en premier lieu de quelques blocs **System.out.println** pour afficher dans la console les informations vis-à-vis des bases de données ainsi que comment y accéder.

On y trouve ensuite un bloc try dans lequel on initie un scanner, pour que l'utilisateur puisse interagir avec la console, qui est importé grâce à la bibliothèque **java.util.Scanner**. L'utilisateur est ensuite invité à saisir un entier, récupéré grâce à la méthode : **scanner.nextInt()**, qui permet de transformer ce qu'écrit l'utilisateur en entier. On entre cet entier dans un **switch case**, pour que le programme réagisse en fonction de ce que l'utilisateur choisit.

Premier cas : l'utilisateur souhaite accéder à la Base de donnée locale et entre 0. Se lance donc la méthode **main()** de la classe **MainLoc**, qui gère toute la partie locale, qui sera expliquée plus tard. Lorsque l'utilisateur a terminé et quitte la Base de donnée locale, on fait un **return** afin de fermer le programme

Deuxième cas : cette fois-ci l'utilisateur a entré 1 et veut accéder aux bases de données distantes. On appelle dans ce cas la classe **Interface** et sa méthode **create()**, qui, de même, sera expliquée plus tard. Lorsque l'on quitte, on fait un **return** pour fermer le programme.

Troisième cas : l'utilisateur a entré une commande autre, le programme prévient le système qu'une commande inconnue a été entré, et lance **Interface.create()**, les Base de données distantes ayant été désignées comme BDD par défaut.

## II. AUTHENTIFICATION

Cette classe est utilisée pour la gestion des comptes utilisateurs.

Ici , une fois XAMPP installer et démarrer, lancer mySql et Apache lancer. Le programme vérifie au lancement s'il la base de données utilisateurs existe et si ce n'est pas le cas, elle la cree avant de poursuivre.

Par la suite, il va pouvoir ajouter un utilisateur ou vérifier si les informations saisies pour se connecter sont bonnes. Pour cela il a deux méthodes qui retournes des booleans « **inscrire** » et « **connecter** » qui prennent en paramètres 2 Strings(le pseudo et le mot de passe) et un **JLabel** qui sert à l'affichage des erreurs Sql ou de connexion dans l'interface graphique .Ce Label d est importer via « import javax.swing.JLabel », de plus pour un souci de mise en page, le texte est afficher en rouge pour cela « java.awt.Color» est importer et utiliser par le « setForeground ».Elles utilisent des **PreparedStatement** qui permettent d'enregister une syntaxe de code sql et de modifier les « ? » par les valeurs concernées plus tard via leur méthode **setString(int,String)** où int est l'ordre d'arrivé de la valeur dans la requête de la gauche vers la droite. Ici en cas d'erreur via le **catch (SQLException)** on retourne directement l'erreur sql tel quel en cas de non respect d'une contraintes d'intégrité sql définie.

### III. Satellite

Classe utilisée pour les bases de donnée distante, elle possède 3 attributs, privés pour garantir l'encapsulation des données (les informations des satellites), un constructeur servant à l'instanciation des objets, un **toString** pour retranscrire toutes les informations des satellites en String et un **getter** par attributs, pour pouvoir les récupérer individuellement tout en respectant le principe d'encapsulation

Elle représente un satellite au sein de la partie graphique de l'application. Elle sert à modéliser les données manipulées par le programme, notamment lors de la récupération d'informations depuis différentes sources internet.

## IV. Interface

La classe **Interface** gère l'ensemble de l'interface graphique de l'application SeeStar. Elle permet à l'utilisateur d'interagir avec les bases de données distantes via plusieurs fenêtres développées avec la bibliothèque **Swing**.

La classe repose sur différents éléments statiques :

Des objets **JFrame** représentant les différentes pages (page d'accueil, page principale des bases distantes, pages spécifiques),  
des **ActionListener** pour gérer la navigation entre les pages.

Leur état statique permet d'y avoir accès facilement depuis n'importe où dans le code et de pouvoir les utiliser à différents endroits sans problème d'existence à différents endroits.

**create()** initialise l'ensemble de l'interface graphique en appelant les méthodes de création de chaque fenêtre. Les appels sont effectués via : **SwingUtilities.invokeLater(...)** Cela garantit que l'interface est exécutée dans le thread graphique (Event Dispatch Thread),

La navigation entre les différentes pages est assurée par des **ActionListener**, importés par la bibliothèque awt qui gère les événements liés aux boutons, qui modifient la visibilité des fenêtres à l'aide de la méthode **setVisible(true/false)**. De cette manière, on simule un changement de page.

Dans chacune des pages, on va retrouver les mêmes 4 premières lignes :

1. On crée la fenêtre avec son titre
2. On précise la manière de fermer la fenêtre avec **setDefaultCloseOperation()** avec le paramètre **JFrame.EXIT\_ON\_CLOSE**, pour fermer la page en appuyant sur la croix rouge.
3. On définit la taille de la fenêtre avec **setSize()**
4. On place la fenêtre au centre avec **setLocationRelativeTo(null)**

Pour la page d'accueil :

On fait les 4 étapes de création de la page.

On vient instancier un **JPanel**, qui va ranger les éléments de notre page, puis on créer des **JLabel**, c'est-à-dire des textes qui seront affichés, des **JTextField** à remplir, un **JTextField** où le texte est visible pour le pseudo, et un **JPasswordField** où le texte est visible sous forme de point pour le mot de passe, et quelques boutons d'inscription et de connexion, dont on modifie la couleur de fond, du texte,

et l'emplacement. Enfin, on créer le bouton qui mènera à la BDD distante, et le bouton qui servira dans le futur pour une interface graphique de la BDD locale.

Après avoir créé les éléments, on les organise. On définit le panel avec un **GridLayout()**, auquel on ajoute les éléments un à un, dont un **JPanel** qui met les boutons côte à côte sur la page, et on ajoute le panel à la page et enfin, on affiche la page.

On créer les **ActionListener** spécifiques aux boutons de connexion et d'inscription.

On instancie une **Authentication A**, un **String name** venant du **JtextField Pseudo** et un **String ChampMDP** venant du **JTextField ChampMDP**. A l'aide de **A**, on vérifie si l'utilisateur se connecte ou s'inscrit, et si il s'inscrit pour la première fois ou se connecte bien, alors on utilise **remove()** pour effacer les éléments dans le panel, on **revalidate()** et **repaint()** pour assurer qu'il n'y ai pas de problèmes d'affichage et on remplit avec cette fois le **JPanel** d'accueil de base, celui avec le pseudo de l'utilisateur et les boutons d'accès aux différentes BDD. Sinon, un message d'erreur s'affiche.

Pour la page d'accueille des BDD distantes :

On fait les 4 étapes de création de la page.

On créer les éléments de la page : un **JLabel** qui contient du HTML pour la mise en page (Java utilise du HTML pour cela) pour mettre sur plusieurs lignes si la case est trop petite, que l'on placera au centre de sa case avec **SwingConstants.CENTER**, on en colorie le texte en blanc. On crée 2 **JPanel**, un pour le haut de la page (1 ligne, 1 colonne) et un pour le bas (1 ligne, 3 colonnes), puis les boutons, qui mèneront aux différentes pages avec **addActionListener()**.

On organise ensuite la page en 2 lignes et 1 colonne avec **setLayout(new GridLayout(2, 1))**. On met le **JLabel** dans le **JPanel** du haut, et les boutons dans le **JPanel** du bas, leur couleur de fond est modifiée, et on rajoute les **Jpanel** dans la

fenêtre. Le choix de faire 2 **JPanel**, bien qu'un seul soit suffisant, est pour que le code reste clair, et peut permettre la préparation d'ajout de fonctionnalités futures.

Pour les 3 pages affichant les informations des Satellite, leur organisation est identique, on vient juste modifier le type de satellites et l'URL du site où l'on va chercher des informations.

On fait les 4 premières étapes de création de fenêtres.

Puis on instancie un **JLabel** présentant le type de satellites voulus. Dans un **String**, on stocke l'url du site où on va prélever les informations, puis on crée une **ArrayList**, importée par **java.util.ArrayList**, qui va venir stocker des **Satellite** en utilisant la méthode **telecharger()** avec l'url en paramètre.

Cette méthode récupère les données sous forme de TLE de la page Web qui se présentent sous cette forme:

```
(OBJECT_NAME
OBJE OBJECT_NAME,OBJECT_ID,EPOCH....
CLASSIFICATION_TYPE.....)
```

Il s'agit d'une série de données sur les satellites. Ici nous nous limitons à 3 d'entre eux: le nom en **ligne 0**, l'id dans la **ligne 1** et l'inclinaison dans la **ligne 2**

Après avoir récupéré chacune de ces lignes pour pouvoir récupérer avec précision l'information souhaitée, nous utilisons des **Ligne(i).substring(m, n).trim();**

Elle permet de récupérer entre le texte entre les caractères m et n de la ligne i. Une fois cela fait, ces données sont envoyées dans l'ArrayList.

Pour l'affichage, on instancie un tableau de **String colonnes** contenant le nom des informations des **Satellite**. Ensuite, un **DefaultTableModel** est instancié, avec en paramètre colonnes et 0, pour les colonnes du tableau et le nombre de lignes. Ce **DefaultTableModel** est ensuite utilisé pour instancier un **JTable**, soit un tableau, où l'on affichera nos **Satellite**. Dans une boucle, on vient remplir le **JTable** avec des **Object[]** contenant les nom, id et inclinaison des **Satellite**. On instancie ensuite un **JScrollPane** avec en paramètre le **JTable**, pour faire défiler les informations. Pour centrer les informations dans les colonnes, on crée un **DefaultTableCellRenderer**, puis on permet le tri des lignes en fonction de sur quelles colonnes on clique avec **setAutoCreateRowSorter(true)**.

Enfin, on vient organiser la page en 2 lignes. La première ligne contiendra 2 colonnes grâce à un **JPanel**. La première colonne contiendra le **JLabel** de



présentation des informations, et la deuxième différents boutons pour voyager entre les autres pages d'informations ou revenir à la page d'accueil, espacé par un espace rempli d'un **JLabel** vide. Dans le **JPanel** du bas, on ajoute le **JScrollPane** contenant les informations souhaitées. On met les **JPanel** dans la fenêtre, à laquelle on applique **revalidate()** et **repaint()**.

Et ceci pour chacune des 3 pages : météo, StarLink et GPS.

vous devrez importer `sqlite-jdbc-3.53.0.0.jar` pour pouvoir par la suite utiliser toutes les librairies nécessaires au bon fonctionnement du programme

vous devrez import

```
• java.sql.SQLException  
• java.util.Scanner  
• java.sql.Statement  
• java.sql.ResultSet  
• java.sql.Connection  
• java.sql.DriverManager
```

Ces bibliothèques sont disposées dans les différentes classes de la partie locale du projet.

## V. MainLoc

La classe MainLOC est le point d'entrée de la partie locale du projet. Elle est responsable de l'initialisation des éléments essentiels au fonctionnement du programme, notamment la création du scanner et la définition de l'URL de la base de données SQLite. Cette URL est un élément fondamental du fonctionnement SQL : elle indique au programme où se trouve la base de données à utiliser.

Une fois l'URL définie et le scanner initialisé, la classe Main utilise une boucle `while (true)` pour permettre de relancer automatiquement le programme en cas d'erreur SQL.

Le cœur du programme se trouve dans un bloc `try`, où la méthode `Start.debut(scanner, url);` est appelée. Cette méthode rentre en paramètre scanner qui permettra donc d'utiliser le scanner sans avoir à en recréer un à chaque utilisation et a aussi l'**URL SQLite**, pour établir la connexion SQL dans la classe Start.

Si une erreur SQL survient (par exemple une requête incorrecte, une table inexistante, un problème d'accès au fichier .db), elle est interceptée par le `catch (SQLException e)` et affiche en rouge l'erreur liée au bug dans la console grâce à `System.err.println`, ce qui permet de distinguer les erreurs SQL des messages normaux.

Par la suite, l'utilisateur pourra alors choisir de relancer le programme ou de le quitter à l'aide d'une commande faite dans la console détectée à l'aide d'un scanner, le traitement de l'information se fait avec un simple if / else (si l'utilisateur décide de quitter il s'effectue un simple `break;` qui permet de quitter la boucle while .

Si l'utilisateur décide de quitter le programme alors le scanner se fermera avec `scanner.close()`

bibliothèque :

```
import java.sql.SQLException;
import java.util.Scanner;
```

## VI. Start

La classe Start est une classe qui contient simplement une seule méthode qui est `static void debut(Scanner scanner, String url) throws SQLException`, elle a été défini en static pour pouvoir l'utiliser sans avoir à créer d'objet Start et a pour paramètre un scanner pour ne pas avoir à en créer un et l'url défini dans le MainLoc. Le throw SQLException permet de ne pas traiter dans cette méthode les erreurs SQL.

La classe start utilise une boucle `while (true)` pour permettre de relancer automatiquement le programme quand l'utilisateur a fini sa commande.

`debut` est le menu principal de la partie locale. Elle permet de créer la connexion SQL avec `Connection conn = DriverManager.getConnection(url);` et le Statement avec `Statement stmt = conn.createStatement();`, qui est l'objet permettant d'exécuter des requêtes SQL.

Par la suite il va suivre une suite de sysout pour expliquer à l'utilisateur quoi faire comme commande, détecté avec le scanner, une fois la commande faite il y aura un switch case permettant de savoir quel choix il a fait.

Toutes les classes qui exécutent des requêtes SQL (Sqlitecmd, Commandes\_guide, Affichage\_satellite) reçoivent ce `(scanner, stmt);` en paramètre. Selon le choix de l'utilisateur, Start redirige vers :

`Sqlitecmd.commandeseuls(scanner, stmt);` qui renvoie vers une class qui vas permettre de faire des commandes sql depuis le terminal de commande.

`Commandes_guide.commandesGuide(scanner, stmt);` qui renvoie vers une class qui permet à l'utilisateur d'interagir avec la base sans écrire une seule ligne de SQL

`exit`, qui a un return pour quitter la methode début. on utilise pas un break car un break ne fait que sortir d'une boucle.

Après chaque action, la classe ferme les ressources SQL avec `stmt.close();` et `conn.close();`

## VII. Sqlitecmd

La classe Sqlitecmd correspond au terminal SQL manuel. Elle est encadrée par un while true et permet à l'utilisateur d'écrire directement une requête SQL complète, comme dans un vrai terminal SQLite.

La classe `Sqlitecmd` est une classe qui contient simplement une seule méthode qui est `static void commandeseuls(Scanner scanner, Statement stmt) throws SQLException` ici on a en paramètre un scanner et un Statement pour éviter simplement d'avoir à en recréer ce qui permet d'utiliser les commandes sql et de faire des interactions avec l'utilisateur. La méthode commence par demander à l'utilisateur saisir une requête, dans un String appelé `"executequery"`, dans un premier temps en sélectionnant l'ensemble des éléments présents dans la bdd, cette requête sera exécuté depuis `ResultSet rs = stmt.executeQuery(executequery);` qui enregistre le résultat de la commande et dans un second temps l'utilisateur devras choisir ce qu'il veut afficher et cette demande sera stocké dans `nextaffichage`.

Après avoir fait ça on fait `while (rs.next())` fait qui permet de simplement de faire un "tant qu' il reste une ligne à afficher faire `System.out.println(rs.getString(nextaffichage));`" qui permet

d'afficher le contenu du tableau enregistré dans le rs en affichant seulement l'affichage choisi par l'utilisateur.

`rs.close();` elle permet de **fermer le ResultSet** qui contient le résultat d'une requête SQL

Une fois la commande finie l'utilisateur repars dans le menu qui est dans le start

A savoir que `commandeseuls` est `static` pour permettre son utilisation sans avoir à créer un objet Sqlitecmd.

## VIII. Commandes guide

La classe **Commandes\_guide** représente le mode de navigation guidée dans la base de données locale.

Contrairement au terminal SQL manuel, cette classe permet à l'utilisateur d'interagir avec la base sans écrire une seule ligne de SQL, elle se charge elle-même de construire les requêtes en fonction des choix effectués dans la console.

Elle repose sur une méthode principale, `static void commandesGuide(Scanner clavier, Statement requeteSQL) throws SQLException`, qui contient 2 paramètres `clavier` est simplement le scanner qui est importé, `requeteSQL` est le Statement de cette méthode et `throws SQLException` est la pour rediriger vers le MainLOC les erreurs SQL.

Cette méthode fonctionne dans une boucle `while (true)` afin de permettre à l'utilisateur d'enchaîner plusieurs recherches sans revenir au menu principal. La première étape après le while consiste à demander à l'utilisateur s'il souhaite filtrer les satellites par rapport à leurs nom, leurs opérateurs et les utilisateurs. ensuite la variable

`demandesate` récupère ce texte avec `String demandesate = clavier.nextLine().trim();`. La méthode construit ensuite une condition SQL avec les informations saisies par l'utilisateur. avec

```
conditionFiltre = "WHERE [Operator_Owner;] LIKE '%" + demandesate + "%' "
+ "OR [Name_of_Satellite;] LIKE '%" + demandesate + "%' "
+ "OR [Users;] LIKE '%" + demandesate + "%'";
```

Si l'utilisateur n'a rien saisi ou a saisi \*, la variable `conditionFiltre` reste vide grâce à ce if qui vérifie si ce qu'il a écrit `if (!demandesate.equals("") && !demandesate.equals("*"))`

L'étape suivante consiste à demander combien de satellites l'utilisateur veut afficher. La variable `limite` est initialisée à **10**, qui sert de valeur par défaut. La saisie est entourée d'un `try { / catch (Exception e)`, car l'utilisateur peut entrer autre chose qu'un entier. Dans ce cas le `catch` intercepte l'erreur et affiche un message d'erreur en rouge avec le `err` à la place du `out` dans `System.out.println`. et prend pour valeur par défaut 10 et le buffer du scanner est vidé pour éviter des erreurs avec `clavier.nextLine();`. Si la valeur est inférieure à 0 alors un autre message d'erreur est affiché et `limite` se redéfinit à 10 qui est la valeur par défaut.

Le choix du tri est également sécurisé car la variable `choixAffichage` est initialisée à **1**, ce qui correspond à l'affichage général, une boucle `while (true) {` entoure la demande, cette boucle ne se termine que lorsque l'utilisateur saisit un entier valide et en cas d'erreur le `catch` affiche un message d'erreur, le scanner est vidé et la boucle recommence donc même en cas de saisie incorrecte, le programme ne plante jamais.

Une fois les éléments définis la classe appelle : `Affichage_satellite.afficher(requeteSQL, conditionFiltre, limite, clavier, choixAffichage);` Cette méthode se charge de construire la requête SQL complète avec en paramètre tous les éléments définis, le scanner `clavier` et le statement `requeteSQL`, d'ajouter le filtre si nécessaire, d'ajouter le tri correspondant au choix, d'exécuter la requête via le Statement et d'afficher les résultats. `commandesGuide` ne manipule donc jamais directement SQL : elle délègue toute la partie technique à `Affichage_satellite.afficher`.

fin de chaque recherche, la classe demande à l'utilisateur s'il souhaite recommencer. La variable `recommencer` récupère ce choix. Si l'utilisateur entre `1`, la boucle recommence depuis le début. Si l'utilisateur entre `0`, un return est fait pour sortir de la méthode pour revenir au menu principal. Si l'utilisateur ne rentre pas `0` ou `1` un message d'erreur sera affiché et il sera renvoyé au menu principal du start, si l'erreur est causé par un int différent alors l'action se fera dans le `try {` sinon elle se fera dans le `catch (Exception e)`.

Ce mécanisme permet d'éviter de devoir repasser par le menu principal du start à chaque utilisation, cela permet de rendre l'utilisation de `commandesGuide` plus fluide.

A savoir que `commandesGuide` est `static` pour permettre son utilisation sans avoir à créer un objet `Commandes_guide`.

## IX. Affichage satellite

La classe `Affichage_satellite` est une class est défini static pour pouvoir l'appeler sans avoir a créer un objet `Affichage_satellite`, elle est dédiée à l'affichage des informations provenant de la base de données locale, Elle centralise toute la logique liée à la construction des requêtes SQL, au tri des résultats et à l'affichage complet des satellites, Elle ne gère pas la connexion SQL elle-même car elle reçoit un Statement déjà créé par la classe Start.

La méthode afficher()

```
static void afficher(Statement requeteSQL, String conditionFiltre, int limite, Scanner clavier, int choixAffichage) throws SQLException {
```

est la méthode principale de cette classe, elle reçoit en paramètres un Statement `requeteSQL` pour exécuter les requêtes SQL, une condition SQL `conditionFiltre` qui peut

être vide (filtre), une limite `limite` de résultats, un Scanner `clavier` (pour d'éventuelles interactions), un choix d'affichage `choixAffichage` déterminant le type de tri.

À partir de ces informations, la méthode construit une requête SQL adaptée. Le choix du tri est géré via un switch case, ce qui permet de sélectionner facilement la colonne sur laquelle trier les résultats. Chaque cas correspond à un type d'information (date, puissance, masse, orbite, etc.). Si l'utilisateur ne choisit rien de valide, un affichage général est utilisé par défaut. par la suite quand le case est choisis la commande sql se finalise en l'envoyant dans `commandesqlINVARCHAR` si l'information est de type `NVARCHAR` et `commandesql` si l'information est de type `VARCHAR` après ça elle est enregistré dans `requete` sauf si c'est l'affichage qui est choisis ça, dans ce cas là tout est directement fait dans le case car la commande ne peut pas être faite dans les méthodes `commandesqlINVARCHAR` et `commandesql` et comme elle n'est faite qu'une fois il a été jugé inutile de recréer une méthode pour ça.

Une fois la requête construite, elle est exécutée avec `ResultSet rs = requeteSQL.executeQuery(requete);` le `ResultSet` contient alors toutes les lignes renvoyées par SQLite.

La méthode parcourt ces lignes une par une avec `while (rs.next()) {` et appelle `afficherSatelliteComple(rs);` pour afficher toutes les informations du satellite et un compteur est lancé pour savoir combien de satellites on été affiché, si il n'y a aucun satellite qui a été affiché alors il sera affiché `Aucun satellite trouvé !` grâce à ce if `if (compteur == 0) {`  
`System.out.println("Aucun satellite trouvé !"); }`

À la fin, `rs.close();` est utilisé pour fermer proprement le tableau de résultats et libérer les ressources SQL.



## Méthodes de construction SQL : `commandesqlNVARCHAR()` et `commandesql()`

La classe utilise deux méthodes pour générer les requêtes SQL selon le type de colonne.

`static String commandesqlNVARCHAR(int limite, String conditionFiltre, String choix)` est utilisée pour les colonnes contenant des nombres mais stockées en texte (NVARCHAR). Pour trier correctement ces valeurs, la méthode utilise `CAST(" + choix + " AS REAL)` cela permet à SQLite de convertir les valeurs textuelles en nombres avant de les trier, ce qui évite les tris incorrects.

`static String commandesql(int limite, String conditionFiltre, String choix)` est utilisée pour les colonnes de texte classiques (VARCHAR). Le tri se fait simplement avec `ORDER BY " + choix + "ASC"`

Les deux méthodes ajoutent également la condition SQL différente mais qui garde la même structure si aucun filtre supplémentaire a été fourni pour éviter les bug sql.

## Affichage complet : `afficherSatelliteComplet()`

`static void afficherSatelliteComplet(ResultSet rs) throws SQLException`

Cette méthode affiche toutes les colonnes d'un satellite de manière claire et structurée, elle ne prend qu'un `ResultSet` en paramètre. (celui fait dans la méthode `afficher`)

Elle utilise `rs.getString("ce que l'on veut afficher;")` pour récupérer chaque champ du `ResultSet` et les afficher dans un format lisible en utilisant la même méthode d'affichage que `commandeseuls` à la différence qu'ici est que ce que l'on veut affiché est prédéterminé et ne demande donc pas de connaître la table sql, elle ne contient aucune logique SQL, elle se contente d'extraire les données déjà récupérées et de les présenter proprement.

savoir que toutes les méthodes de `Affichage_satellite` sont `static` pour permettre leur utilisation sans avoir à créer un objet `Affichage_satellite` et `afficher` et `afficherSatelliteComple` utilisent `throws SQLException` pour demander à MainLoc de gérer les erreurs QSL à leurs place. `commandesql` et `commandesqlINVARCHAR` n'en n'ont pas besoin car ils ne font techniquement que remplir un String.